

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

I GNANA PRIYAN G(192521393) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Debugging & Optimisation Task

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)

A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

- A. Identify and fix logical and implementation errors in key scheduling and permutation steps.
- B. Optimize the code to reduce execution time for large input datasets (≥ 10 MB).
- C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

2. Performance Analysis of Block Cipher Modes of Operation

A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

- A. Debug incorrect padding and IV handling causing decryption failures.
- B. Refactor the code to minimize redundant computations and memory usage.
- C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

3. Debugging RSA Cryptosystem Implementation

A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

- A. Identify issues in prime generation, key generation, and modular exponentiation.
- B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
- C. Evaluate the trade-offs between key size, security, and computational performance.

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange

A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

- A. Debug incorrect shared key generation due to improper modular arithmetic.
- B. Optimize exponentiation operations for large prime values.
- C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation

A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

- A. Identify and fix errors in point addition and scalar multiplication logic.
- B. Optimize the implementation using efficient algorithms (e.g., double-and-add).

- C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

Assessment Tool 3 — Debugging & Optimisation Task

Question 4: Optimization and Security Analysis of Diffie-Hellman Key Exchange

Scenario: A Python implementation of Diffie-Hellman key exchange produces incorrect shared keys and performs poorly for large primes.

A. Debugging the Shared Key Generation (Improper Modular Arithmetic)

Buggy code — the most common mistake is computing the exponentiation without reducing modulo p (using `**` instead of the modular form), which produces astronomically large numbers and, worse, a shared secret that does not actually match between the two parties if any intermediate value is truncated or handled inconsistently:

```
# --- BUGGY VERSION ---
def generate_public_key(base, private_key, prime):
    return base ** private_key          # BUG 1: no mod reduction

def generate_shared_key(other_public, private_key, prime):
    return other_public ** private_key  # BUG 2: no mod reduction

p = 23
g = 5
a_priv = 6
b_priv = 15
A = generate_public_key(g, a_priv, p)  # huge int, not mod p
B = generate_public_key(g, b_priv, p)
key_a = generate_shared_key(B, a_priv, p)
key_b = generate_shared_key(A, b_priv, p)
print(key_a == key_b)                  # False -- keys do not match
```

Root cause: Diffie-Hellman is defined entirely over modular arithmetic — every exponentiation must be reduced mod p at each stage: $A = g^a \bmod p$, $B = g^b \bmod p$, and the shared key $= B^a \bmod p = A^b \bmod p = g^{ab} \bmod p$. Dropping the "mod p " breaks the algebraic identity that makes both sides converge to the same value, and also causes the numbers to explode in size for large exponents.

Fixed code:

```
# --- FIXED VERSION ---
def generate_public_key(base, private_key, prime):
    return pow(base, private_key, prime)  # FIX: mod p applied

def generate_shared_key(other_public, private_key, prime):
    return pow(other_public, private_key, prime) # FIX: mod p applied

p = 23
g = 5
a_priv = 6
b_priv = 15
A = generate_public_key(g, a_priv, p)
B = generate_public_key(g, b_priv, p)
key_a = generate_shared_key(B, a_priv, p)
key_b = generate_shared_key(A, b_priv, p)
print(key_a == key_b)                  # True -- shared keys match
```

B. Optimizing Exponentiation for Large Prime Values

1. Use Python's built-in three-argument `pow(base, exp, mod)` instead of computing `base**exp` first

and taking mod afterwards. The three-argument form uses fast modular exponentiation (square-and-multiply) internally, written in C, giving $O(\log n)$ multiplications instead of computing a huge intermediate integer.

2. Avoid re-deriving the modulus separately ($x = \text{base}^{**}\text{exp}$; $x \% \text{mod}$) — this computes an enormous intermediate value for large exponents (which is exactly what caused the slowdown) before reducing it, wasting both time and memory.
3. Use a cryptographically secure random generator (`secrets.randbelow(p-2) + 1`) for private keys instead of the default random module, since predictable randomness is itself a security bottleneck.
4. For repeated key exchanges, cache/reuse the safe prime p and generator g rather than regenerating them, since safe-prime generation is the most expensive part of DH setup.

Illustrative optimization:

```
import secrets

def generate_private_key(prime):
    return secrets.randbelow(prime - 2) + 1    # secure RNG

# Fast modular exponentiation (what pow(a, b, m) already does internally)
def fast_pow_mod(base, exp, mod):
    result = 1
    base %= mod
    while exp > 0:
        if exp & 1:
            result = (result * base) % mod
        base = (base * base) % mod
        exp >>= 1
    return result
```

This reduces the exponentiation cost from potentially exponential blow-up (unreduced huge integers) to $O(\log(\text{exp}))$ modular multiplications, which is essential once p is a realistic 2048-bit prime rather than a toy value like 23.

C. Mitigating Man-in-the-Middle Attacks and Performance Impact

Security threat: Plain Diffie-Hellman has no authentication — it only guarantees that two parties agree on a shared secret, not who they are agreeing with. An attacker (Mallory) can intercept A and B, and run two separate DH exchanges (one with Alice, one with Bob), decrypting and re-encrypting all traffic while both victims believe they have a private channel.

5. Mitigation — Authenticated (Signed) Diffie-Hellman: Each party digitally signs their public value (A or B) using a long-term private key (RSA or ECDSA) whose corresponding public key is verified via a certificate (or a pre-shared trusted key). The receiver verifies the signature before accepting the public value, so a MITM cannot forge a valid public key without the victim's private signing key.
6. This is essentially what TLS does (DHE/ECDHE cipher suites combined with the server's certificate signature) — it is called Authenticated Diffie-Hellman or Station-to-Station (STS) protocol.

Extended code sketch (conceptual):

```
# Alice signs her DH public value with her long-term private signing key
signature_A = sign(private_signing_key_alice, A)
send(A, signature_A, certificate_alice)

# Bob verifies before trusting A
if verify(certificate_alice.public_key, A, signature_A):
    proceed_with_key_exchange()
else:
    abort('Possible MITM -- signature invalid')
```

Performance impact: Adding signing/verification introduces extra asymmetric operations (one signature generation and one verification per party per session). For RSA-2048 signatures this typically adds a few milliseconds per exchange; for ECDSA it is even cheaper. This overhead is negligible compared to the security gained, especially since the DH exponentiation itself (after

optimization with fast modular exponentiation) is already the dominant, unavoidable cost. In short: authentication adds a small, constant, justified performance cost in exchange for closing the MITM vulnerability.

Question 5: Debugging and Enhancing Elliptic Curve Cryptography (ECC)

Scenario: A Python ECC implementation for key exchange produces incorrect results for certain curve parameters.

A. Identifying and Fixing Errors in Point Addition and Scalar Multiplication

Buggy code — typical errors include: not handling the point-at-infinity case, using the wrong slope formula when adding a point to itself ($P = Q$) versus adding two distinct points ($P \neq Q$), and forgetting to compute the modular inverse correctly:

```
# --- BUGGY VERSION ---
def point_add(P, Q, a, p):
    x1, y1 = P
    x2, y2 = Q
    # BUG 1: always uses the 'distinct points' formula,
    #         even when P == Q (should use the tangent/doubling formula)
    m = (y2 - y1) / (x2 - x1)          # BUG 2: '/' is float division,
                                     #         not modular inverse!

    x3 = m**2 - x1 - x2
    y3 = m * (x1 - x3) - y1
    return (x3 % p, y3 % p)
```

Root causes:

7. Point doubling vs. addition: When $P == Q$, the slope must be computed with the tangent formula $m = (3x_1^2 + a) * \text{inverse}(2y_1, p) \bmod p$, not the secant formula $(y_2 - y_1) / (x_2 - x_1)$. Using the wrong formula silently gives an incorrect resulting point.
8. Division must be modular inverse: In finite-field arithmetic there is no real division. '/' must be replaced with modular inverse computed via the Extended Euclidean Algorithm or $\text{pow}(d, p-2, p)$ (valid when p is prime, by Fermat's Little Theorem).
9. Point at infinity (identity element) is not handled: if P is the identity, the result should just be Q (and vice versa); if $x_1 == x_2$ and $y_1 == -y_2 \bmod p$, the result should be the point at infinity. Skipping these checks causes crashes or silently wrong output for edge-case inputs — exactly the "certain curve parameters" described in the scenario.

Fixed code:

```
# --- FIXED VERSION ---
def inverse_mod(k, p):
    return pow(k, p - 2, p)          # modular inverse (p prime)

def point_add(P, Q, a, p):
    if P is None: return Q          # identity element handling
    if Q is None: return P
    x1, y1 = P
    x2, y2 = Q
    if x1 == x2 and (y1 + y2) % p == 0:
        return None                 # result is point at infinity
    if P == Q:
        # Point doubling (tangent line)
        m = ((3 * x1 * x1 + a) * inverse_mod(2 * y1, p)) % p
    else:
        # Point addition (secant line)
        m = ((y2 - y1) * inverse_mod(x2 - x1, p)) % p
    x3 = (m * m - x1 - x2) % p
    y3 = (m * (x1 - x3) - y1) % p
    return (x3, y3)
```

B. Optimizing Scalar Multiplication (Double-and-Add)

Naively computing $k \cdot P$ by adding P to itself k times is $O(k)$ point additions — infeasible for real key sizes ($k \sim 256$ -bit). The standard optimization is the double-and-add algorithm, which mirrors fast modular exponentiation and runs in $O(\log k)$ point operations:

```
def scalar_multiply(k, P, a, p):
    result = None          # point at infinity (identity)
    addend = P
    while k:
        if k & 1:
            result = point_add(result, addend, a, p)
            addend = point_add(addend, addend, a, p)  # doubling
            k >>= 1
    return result
```

10. This reduces scalar multiplication from thousands/millions of additions to roughly $\log_2(k)$ doublings plus additions — e.g., for a 256-bit scalar, at most ~ 256 doublings and ~ 128 additions on average, instead of up to 2^{256} additions.
11. Further real-world optimizations (beyond this scope but worth noting): using Jacobian/projective coordinates to avoid expensive modular inversions on every step, windowed NAF (non-adjacent form) methods, and constant-time implementations to prevent timing side-channel attacks.

C. ECC vs RSA — Efficiency, Security Strength, and Suitability for IoT

Aspect	RSA	ECC
Security basis	Integer factorization problem	Elliptic curve discrete logarithm problem (ECDLP)
Key size for ~ 128 -bit security	~ 3072 -bit key	~ 256 -bit key
Computation cost (encrypt/verify)	Relatively fast with small public exponent	Fast, but generally requires point multiplication
Computation cost (decrypt/sign)	Expensive — large modular exponentiation with big private key	Much cheaper for equivalent security due to smaller key size
Memory / bandwidth footprint	Large keys and ciphertext/signature sizes	Small keys, small signatures/ciphertext — ideal for constrained links
Power consumption	Higher, due to large-number arithmetic	Lower — well suited to battery-powered devices

Justification for resource-constrained (IoT) environments: IoT devices in a smart home system typically have limited CPU power, small memory, and run on batteries, while also needing to transmit data over low-bandwidth wireless links. Because ECC achieves the same security level as RSA with roughly 10x-12x smaller keys (e.g., 256-bit ECC $\approx$ 3072-bit RSA in strength), it requires far less computation, memory, storage, and transmitted data for keys and signatures. This directly translates into faster operations, lower power/battery drain, and reduced bandwidth usage — all critical constraints for IoT.

Recommendation: ECC (specifically ECDH for key exchange and ECDSA for authentication) is the more suitable cryptographic approach for the smart home IoT scenario. RSA remains a reasonable choice only where legacy compatibility is required or where the devices are not significantly power/bandwidth constrained (e.g., a home hub with mains power and stronger hardware), but for the low-power end devices described in the scenario, ECC is clearly the better trade-off between security strength and computational efficiency.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

